

ROSE: Rollout On Serving GPUs via Cooperative Elasticity for Agentic RL

Wei Gao^{†*}, Yuheng Zhao^{†*}, Dilxat Muhtar[‡], Dakai An[†], Xuchun Shang[‡],
Tianyuan Wu[†], Lunxi Cao[†], Shaopan Xiong[‡], Weixun Wang[‡], Ju Huang[‡],
Teng Ma[‡], Siran Yang[‡], Jiamang Wang[‡], Lin Qu[‡], Bo Zheng[‡], Wei Wang[†]

[†]HKUST

[‡]Alibaba Group

*Equal contribution

Abstract

Agentic reinforcement learning (RL) has emerged as a key driver for improving LLMs’ multi-step reasoning and tool-use capabilities. However, its efficiency is bottlenecked by long-tail rollouts with multi-turn environment interactions. This renders static GPU provisioning a poor fit: overprovisioning wastes GPUs on stragglers, whereas underprovisioning increases contention and slows training.

We observe that production serving clusters routinely leave substantial GPU compute and memory headroom. We therefore argue for *cooperative elasticity*: opportunistically repurposing underutilized serving GPUs to execute rollouts. Realizing cooperative elasticity is non-trivial due to preserving serving Service Level Objectives (SLOs) under bursty traffic, and heavy communication overhead. We present *ROSE*, a cooperative, resource-elastic post-training system that safely harvests idle compute and memory on serving GPUs to expedite agentic RL rollouts. *ROSE* comprises three components: (1) an SLO-safe co-serving executor that improves rollout throughput while preserving serving SLOs via efficient GPU memory and compute sharing; (2) a cross-cluster weight transfer engine that leverages weight shards and sparsity for fast cross-cluster weight synchronization; and (3) an elastic rollout scheduler that dynamically provisions cooperative capacity and routes trajectory rollouts across dedicated rollout GPUs and opportunistic serving GPUs. Experiments across multiple model sizes and cluster scales show that *ROSE* improves average end-to-end throughput by 1.20–3.31× versus state-of-the-art resource-fixed and elastic baselines.

1 Introduction

The advent of agentic reinforcement learning (RL) is reshaping large language model (LLM) post-training, shifting models from passive response generation to actively interacting with complex, dynamic environments. Recent advances in tool use [20, 68], computer use [34, 36, 38], and software engineering [26, 61, 77] suggest that this training paradigm enables LLMs to progressively solve complex tasks through planning [31], tool use [59], and multi-step reasoning [48].

A typical agentic RL training pipeline consists of two stages, *rollout* and *training*. During rollout, the agent LLM interacts with environments over multiple turns. At each turn, the agent *decodes* tokens that serve as action signals,

and then runs *prefill* on the feedback returned by the environment. The resulting sequence of actions and feedback forms a trajectory. During training, the agent LLM updates its weights using the collected trajectories, and then synchronizes the updated weights to the rollout stage for the next iteration. *Practically*, agentic RL is predominantly pursued in industry, where organizations can absorb the substantial training cost and the operational complexity of interactive environments, and where sizeable serving GPUs are already deployed for production usage of agentic LLMs.

Similar to conventional single-turn RL, end-to-end agentic RL training is dominated by the rollout stage, which accounts for over 70% of total wall-clock time (see Figure 1a). Rollout batches also exhibit a pronounced long-tail distribution in execution time (see Figure 1b). Beyond this shared characterization, agentic RL rollouts differ from single-turn RL in two key aspects. First, multi-turn environment interactions impose stringent demands on compute throughput to reduce rollout overhead. In particular, the prefill phase is compute-intensive and benefits from prefix caching and resource scaling (see Figure 1c). Second, agentic RL typically launches additional rollouts to improve trajectory quality for gradient stability [81, 87], which in turn exacerbates GPU memory pressure and contention. Collectively, both characteristics suggest that scaling out GPUs for rollout can expedite agentic RL training. However, long-tail trajectories render static provisioning inefficient: overprovisioning leaves GPUs idle while waiting for stragglers, whereas underprovisioning amplifies contention among concurrent rollouts and increases rollout latency.

Recently, many *resource-fixed* RL systems have been proposed to improve efficiency *given a fixed GPU budget*, using techniques such as resource disaggregation [69, 89], colocation [24, 55], asynchronous execution [13, 19, 35, 54], fully asynchronous training [13], tail batching [16], stage overlapping [4, 71, 90], and speculative decoding [5, 23, 45, 52]. While effective within a fixed allocation, these systems still require manual GPU provisioning. Because rollout demand varies over time, an allocation that is appropriate for one RL step can be suboptimal for another (Figure 1d).

Existing *resource-elastic* RL systems typically realize elasticity by *acquiring additional* GPUs for rollouts on demand, using spot instances (e.g., RLBoost [70]) or serverless GPUs (e.g., tinker [62], OpenPipe [42]). While effective when spare

capacity is readily available, this add-capacity approach has three limitations. First, the achievable speedup is bounded by the availability of incremental GPUs, which can be scarce during cluster-wide contention [70]. Second, because these GPUs are outside the steady-state deployment, elasticity often incurs repeated model initialization when capacity churns (e.g., spot preemption or serverless pool rotation), reducing effective throughput (Figure 3c). Third, acquiring extra GPUs typically increases monetary cost due to dynamic provisioning and pricing variability.

An alternative is *bidirectional autoscaling*, which shrinks the serving cluster during low load and redirects freed GPUs to rollouts. However, serving traffic is bursty at second-level granularity (§3.2), and reclaiming GPUs back to serving takes tens of seconds (Figure 3c), making bidirectional autoscaling unable to preserve serving SLOs (§3.3, §6.5).

Instead, we explore *cooperative elasticity*, which reuses *already-deployed* serving capacity within an organization. In many industrial LLM companies, the RL training team and the LLM serving team are part of the same institution and operate separate large-scale GPU clusters. Prior studies [44, 66, 74, 82, 85] report that fluctuating serving traffic causes serving GPU underutilization. Our empirical study (see Figure 3b) confirms that serving GPUs have low compute utilization (18.9%) and memory utilization (14.3%). Cooperative elasticity turns this headroom into rollout capacity by harvesting idle GPU cycles and memory from the serving cluster subject to Service Level Objectives (SLOs) constraints. By reusing existing capacity, cooperative elasticity expands effective rollout resources without acquiring additional GPUs, avoids on-demand provisioning overheads, and incurs little incremental monetary cost.

In this paper, we present *ROSE*, a cooperative, elastic system for agentic RL post-training that harvests serving GPUs for rollouts. While promising, repurposing serving GPUs poses two primary challenges. First, co-locating heterogeneous serving and rollout LLMs must preserve serving SLOs under bursty traffic, despite contention for both GPU memory and compute (C1). Prior GPU multiplexing systems cannot simultaneously handle heterogeneous KVC layouts across models [44], retain rollout prefix cache in GPU for short-lived reuse [72, 82], and yield resources promptly under serving bursts [7, 72, 82] (§6.5). We observe three properties of agentic rollouts that enable SLO-safe co-serving: (1) heterogeneous models require flexible KVC rebalancing across incompatible layouts, (2) rollout environment interactions are short-lived (typically <10 s [17]), favoring in-GPU prefix caching with fast memory preemption, and (3) rollouts tolerate seconds-level latency because preempted rollouts can be rerouted to dedicated rollout GPUs. We design a *co-serving executor* that combines CUDA VMM for cross-model KV cache rebalancing, preemptive memory sharing that retains rollout prefix cache in GPU and aggressively reclaims it during serving bursts, and dual-SLO admission control that

prioritizes serving via temporal sharing while opportunistically executing rollouts.

Second, the serving and RL clusters may reside in different datacenters connected by bandwidth-limited links (10–200 Gbps Ethernet), causing cross-cluster weight transfer to take dozens of seconds to minutes (Figure 3d), eroding the speedups from elastic rollouts (C2). Existing bandwidth-optimized communication systems [10, 33, 50, 67] rely on collective communication over fixed process groups with uniform sharding, which cannot accommodate cooperative elasticity where (1) serving GPUs dynamically join or leave across RL steps, and (2) training and serving adopt different parallelism strategies requiring automatic shard mapping. Furthermore, we observe that RL post-training *weight deltas* exhibit lossless sparsity (Figure 6), a property rare in conventional LLM training [40, 49, 76] that can be exploited for cross-cluster transfer. We design a *cross-cluster weight transfer engine* that combines a relay layer for asynchronous, fault-tolerant propagation, shard-aware routing across heterogeneous parallelism configurations, and sparsity-aware compression that transmits only non-zero deltas, reducing transfer overhead to within 20 seconds even under 20 Gbps Ethernet (§6.3).

Beyond these challenges, we design a *rollout scheduler* that capitalizes on cooperative elasticity by dynamically dispatching rollouts across dedicated rollout GPUs and opportunistic serving GPUs. To adapt to time-varying serving load, the scheduler uses two heuristics. First, it performs *turn-wise, concurrency-aware routing* to decide how many rollouts to offload to serving GPUs while maintaining high rollout throughput. Second, it adopts a *cache-affinity placement policy* that preferentially routes each turn to the GPU holding the trajectory’s prefix KVC, and otherwise falls back to load-aware placement. Together, these policies allow *ROSE* to exploit underutilized serving capacity to accelerate rollouts while minimizing interference with serving workloads. This paper makes the following contributions:

- We propose *cooperative elasticity*, the first approach to harvest serving GPUs for RL rollouts while preserving serving SLOs.
- We design a co-serving executor that uniquely supports heterogeneous LLMs, prefix caching, and bursty serving—capabilities not simultaneously addressed by prior co-serving systems.
- We discover that RL weight deltas exhibit >95% sparsity and design the first cross-cluster weight transfer engine that exploits both shard-awareness and sparsity-awareness for efficient RL weight propagation across datacenters.
- We design an elastic rollout scheduler with turn-wise routing and cache-affinity that adapts to time-varying serving load without global synchronization.

- We implement *ROSE* atop ROLL [64] and evaluate it with Qwen3-8B/32B [75] on agentic tasks [9, 58], using 16–48 training GPUs and 16–64 serving GPUs (H800). Compared with resource-fixed baselines ROLL [64] and AReaL [13], *ROSE* improves average throughput by 1.20–3.31× and 1.44–2.69×, respectively. Compared with resource-elastic baselines RLBoost [70] and λ RL [42, 62], *ROSE* outperforms by 1.20–1.26× and 1.29–1.52×, respectively.

2 Background and Motivation

2.1 Agentic RL Training

An agentic RL training pipeline alternates between *rollout* and *training*. In modern agentic RL, rollout is often organized using group-based algorithms [11, 53, 81]. Taking GRPO as an example, the rollout stage launches B_0 *environment groups*, where environments within the same group share the same initial state. For each group, the agent LLM (actor) interacts with the environment over multiple turns: at each turn it observes the current *state*, samples an *action* from its policy, and sends it back to the environment; the environment then transitions to a new state and returns feedback. The agent generates G_0 sampled responses per group, producing G_0 trajectories from the same starting point. When each trajectory receives the terminal signal, a reward worker evaluates it and assigns a scalar reward. In the training stage, the agent LLM updates its model weights using the collected trajectories and reward signals, and the updated weights are synchronized back to the rollout workers for the next step.

2.2 Workload Characterization

We perform workload characterization using Qwen3-8B and Qwen3-32B [75] within the agentic RL framework ROLL [64], configured with a maximum response length of 32k tokens, a batch size of 256, and a group size of 8 under the GRPO algorithm [53], on 16 H800 GPUs. We adopt synchronous RL training to profile the end-to-end training time. Specifically, we run the 8B and 32B LLMs on the agentic tasks FrozenLake and ALFWORLD, respectively, for five consecutive steps.

Dominant Rollout Overhead. We report the performance breakdown in Figure 1a. The end-to-end training time consists of rollout, training, and weight synchronization overhead. The rollout stage accounts for over 70% of total time and dominates end-to-end training. This motivates the tailored rollout optimizations to improve training efficiency.

Long-tail Rollouts. Figure 1b presents the execution time distribution of trajectories during rollout for the 8B and 32B LLMs. We observe a pronounced long-tail pattern: most trajectories finish quickly, while a small fraction take much longer. In particular, the 75th percentile (P75) is at most 30% of the end-to-end rollout time. This observation is consistent with prior studies [16, 35, 65]. Such long-tail phenomena

can cause GPU underutilization, as many GPUs remain idle while waiting for straggler trajectories to complete.

The Impact of Prefill. Multi-turn agentic RL entails frequent prefills [17, 65]. In our workloads, prefill tokens account for 77% (Qwen3-8B) and 86% (Qwen3-32B) of total tokens, making prefill a dominant cost. As a result, prefix caching, which reuses KV cache (KVC) for shared prefixes across requests, is widely used in multi-turn rollouts [12, 17, 35]. To quantify its impact, we vary the number of GPUs for Qwen3-8B and measure average rollout time with and without prefix caching in Figure 1c. Under resource contention, prefix caching improves rollout throughput by up to 3.4× (at 8 GPUs). As more GPUs are allocated, the rollout time reduces from 607 s to 435 s. This contrasts with single-turn RL, where scaling GPUs often yields limited benefit because long-tail samples are dominated by the bandwidth-bound decoding phase. In multi-turn agentic RL, scaling is more effective because of the compute-heavy prefill stage. Next, we analyze how rollout demand varies across steps to understand whether a static GPU allocation can be optimal.

Need for Resource Elasticity. In agentic RL training, agent-environment interaction often yields sparse rewards, and reward values within a group may exhibit low (or even zero) variance, weakening the learning signal. Hence, many algorithmic studies [81, 84, 87] propose redundant sampling: dynamically increasing B_0 to launch additional environment workers, and continuing rollouts until collecting B_0 groups with non-zero reward variance. We observe that many agentic RL jobs enable this feature in our internal cluster. Figure 1d shows that when training a Qwen3-8B model with DAPO [81], the number of generated trajectories vary significantly across steps, with the maximum reaching 5.7× the user specified batch size. With a fixed GPU budget, redundant sampling increases memory pressure, and the high variability in rollout volume makes a static, resource-fixed configuration inefficient. Beyond redundant sampling, even standard GRPO with a fixed batch size benefits from resource elasticity: long-tail rollouts and compute-heavy prefills create variable per-step resource demand, and our end-to-end evaluation (§6.1) confirms that resource elasticity improves average throughput by 1.31–1.46× under GRPO. These results motivate a *resource-elastic* agentic RL system that adjusts rollout GPU allocation over time to reduce contention and shorten step time.

2.3 Limitations of Existing Solutions

Resource-fixed RL systems struggle to balance utilization and training time. Many existing RL training systems [4, 12, 13, 16, 19, 23, 35, 52, 54, 64, 69, 71, 89, 90] provision a fixed number of GPUs for rollouts throughout training. This static allocation cannot align the actual rollout workload demand: rollout latency exhibits heavy tails, so overprovisioning leaves GPUs underutilized due to long-tail trajectories, while underprovisioning increases contention

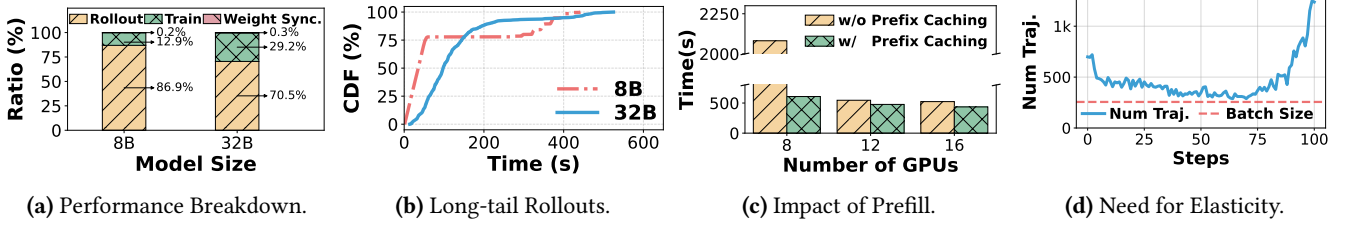


Figure 1. Characterization of agentic RL: (a) The breakdown of end-to-end training time; (b) The long-tail distribution of rollout execution time; (c) The impact of prefill on rollouts; (d) The demand for resource elasticity.



Figure 2. Illustration of Agentic RL Pipeline.

for compute and memory, inflating rollout time. Moreover, as Figure 1d shows, rollout demand varies across steps, making any single fixed allocation suboptimal over time.

Existing elastic RL systems depend on abundant spare GPUs. Existing resource-elastic designs typically rely on spot instances [70] or serverless GPUs [42, 62] to scale rollout capacity on demand. However, as cluster-wide RL demand grows, opportunistic capacity becomes scarce and cannot reliably satisfy rollout demand. As such, the effectiveness of RLBoost [70] is limited by spot availability. Moreover, fluctuating spot capacity introduces frequent resource allocation churn, reducing effective rollout throughput. Similarly, under heavy contention from many concurrent RL jobs, serverless RL systems [42, 62] may trigger frequent reallocation while still failing to secure GPUs for many jobs, undermining elasticity benefits. Beyond, dynamic provisioning exposes training to volatile pricing, increasing and destabilizing the monetary cost of agentic RL training.

3 Opportunities and Challenges

We first outline the opportunity to repurpose serving GPUs for RL via cooperative elasticity. We then quantify the harvestable serving capacity and highlight the key challenges.

3.1 Infrastructure for RL and Serving

Today, many AI organizations operate both RL infrastructure and LLM serving infrastructure, and Figure 4 illustrates a typical deployment within one organization. For clarity, we categorize GPUs into three clusters: a *training cluster*, a *rollout cluster*, and a *serving cluster*. The training and rollout clusters are co-located in the same datacenter and communicate via high-speed interconnects (e.g., NVLink and InfiniBand). The serving cluster may reside in the same or a different datacenter. In large organizations such as Google [27], ByteDance [25], and Alibaba [72], training and serving clusters are routinely placed in separate datacenters for operational isolation, capacity planning, and proximity to end

users. Cross-datacenter links are bandwidth-limited (e.g., 10–200 Gbps Ethernet) [17, 69].

RL infrastructure and LLM serving infrastructure are operated by different teams within the same organization, each managing large-scale GPUs. Since rollout and serving are both inference workloads, both teams often collaborate on optimizing a shared LLM inference engine, a pattern also observed in open-source communities [51]. This organizational structure creates an opportunity for *cooperative elasticity*: *conceptually*, the RL infrastructure team can collaborate with the serving team to *repurpose serving GPUs for rollouts* when rollout demand spikes, thereby enabling resource elasticity without expanding the training cluster.

3.2 Serving Capacity Availability

We next quantify how much serving capacity is *empirically* harvestable for the cooperative elasticity of rollouts.

Fluctuating Serving Traffic Leads to GPU Underutilization. Production LLM serving workloads exhibit fluctuating request rates [47, 63, 66, 74, 82]. Figure 3a plots a 24-hour Microsoft trace [60] at minute granularity alongside three zoomed-in 5-minute windows at per-second granularity. At the minute level, the peak rate reaches 1.7× the 24-hour average. At the second level, burstiness is far more pronounced: per-second peaks reach 4.22×, 1.58×, and 1.73× their respective window averages, consistent with second-level spikes reported by BurstGPT [66]. To absorb such spikes, providers often statically overprovision for peak demand [88], resulting in substantial GPU underutilization. We quantify this by replaying a 24-hour production trace from [47] (preserving original prompt lengths, response lengths, and arrival process) on Qwen3-8B with 8 H800 GPUs. Figure 3b shows the GPU utilization sampled at 1-second intervals and smoothed with a one-minute moving average. On average, GPUs reach only 18.9% SM utilization and 14.3% HBM utilization, confirming that peak-provisioned GPUs remain significantly underutilized for much of the day.

Abundant Serving GPU Capacity for Rollouts. Beyond per-GPU underutilization, serving clusters often comprise thousands of GPUs [72], so even modest slack per GPU aggregates into a large pool of idle cycles and memory. A natural question is whether serving autoscaling [80, 83] can harvest this capacity by shrinking the serving cluster during low

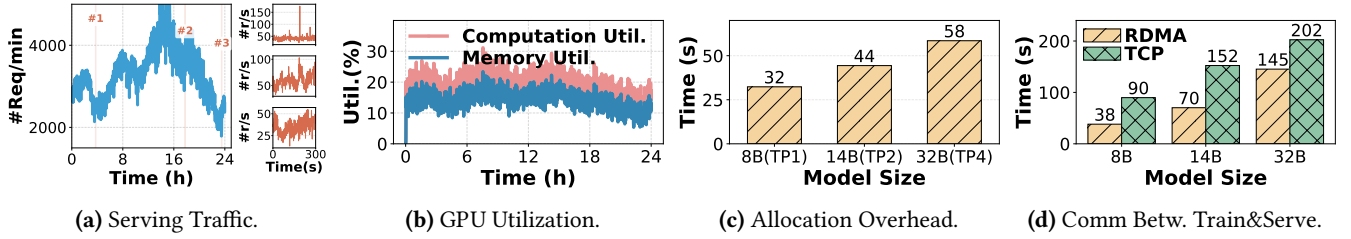


Figure 3. Characterization of serving clusters and workloads: (a) Fluctuating serving traffic; (b) Serving GPU underutilization; (c) High allocation overhead; (d) Substantial communication overhead.

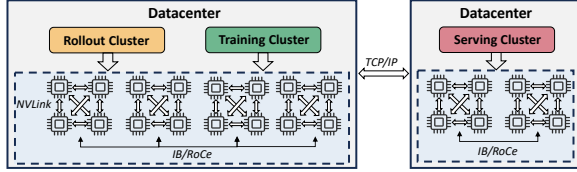


Figure 4. Scheme of Datacenter Infrastructure.

load and redirecting freed GPUs to rollouts. However, bidirectional autoscaling is fundamentally limited: reclaiming GPUs from rollouts back to serving requires evicting in-flight rollouts and reloading models, taking tens of seconds (Figure 3c) and far exceeding typical SLO budgets. Because serving traffic is bursty at second-level granularity, frequent mode switching triggers repeated initialization overhead that erodes throughput gains. Our evaluation (§6.5) confirms that an autoscaling-based baseline cannot safely exploit serving slack without SLO violations. As a result, many providers still adopt fixed-capacity serving deployments to react to bursty traffic, leaving substantial spare capacity that cooperative elasticity can exploit *without* GPU-level mode switching.

3.3 Challenges in Cooperative Elasticity

Making bidirectional autoscaling burst-aware would require preemptive memory sharing and SLO-aware admission control, fundamentally transforming it into a co-serving system. Cooperative elasticity embraces this direction by *co-locating* rollout and serving workloads on the same GPUs. This avoids allocation overhead but requires careful GPU memory and compute sharing to preserve serving SLOs.

C1: Preserving Serving SLOs under Bursty Traffic. Unlike bidirectional autoscaling, which reassigns entire GPUs between workloads, cooperative elasticity keeps both the serving and rollout models resident on each GPU and dynamically shares resources between them. This avoids the tens-of-seconds reallocation overhead (Figure 3c) that makes bidirectional autoscaling impractical, but introduces finer-grained contention for GPU memory and compute.

- **Inefficient memory sharing.** Mainstream LLM serving engines [28, 51] rely on static GPU memory reservation for a given model. A natural co-location approach is to reserve static, exclusive GPU memory regions for the KVC of the serving and rollout LLMs while sharing compute.

This can work under low serving traffic, but bursty arrivals require dynamically resizing the KVC partitions to accommodate more serving requests and meet SLOs. Unfortunately, resizing KVC memory in existing engines typically requires terminating the process and reinitializing it, which can take dozens of seconds (Figure 3c). Moreover, rollouts benefit from prefix caching, which competes with the serving KVC for limited HBM capacity. Under spikes, this contention can inflate decoding latency and trigger SLO violations.

- **Severe compute interference.** LLM serving consists of two phases: *prefill* (compute-intensive) and *decoding* (memory- and latency-sensitive). Serving systems enforce two latency SLOs: *time-to-first-token* (TTFT) and *time-per-output-token* (TPOT), reflecting prefill and decoding latency, respectively. Several systems disaggregate prefill and decoding onto separate GPU pools (PD disaggregation) to stabilize TTFT/TPOT under bursty traffic [43, 88]. Unlike standard multi-tenant serving, where co-located workloads share comparable SLO requirements, rollout inference has fundamentally asymmetric characteristics: rollouts run a *different* LLM with much looser latency targets, generate long multi-turn sequences that sustain GPU occupancy for seconds, and perform both prefill and decode on the same GPU (PD co-location) even when serving deployment uses PD disaggregation. The standard request scheduling cannot bound the compute interference: a single rollout prefill chunk can delay serving decodes, and sustained rollout batches can starve serving prefills (analyzed in §6.2).

C2: Heavy Cross-cluster Communication Overhead. In many deployments, RL training and online serving are provisioned as separate GPU clusters, and weight updates must traverse cross-cluster links via a flexible transfer engine. To quantify this overhead, we measure the end-to-end time to transfer LLM parameters of varying sizes from a GPU node in the RL cluster to a GPU node in the serving cluster using Mooncake Store [46]¹, over TCP (200 Gbps Ethernet) and RDMA (400 Gbps InfiniBand), shown in Figure 3d. Even with InfiniBand (which is uncommon across datacenters),

¹The weight transfer adopted here is a *batch* baseline in §6.3.

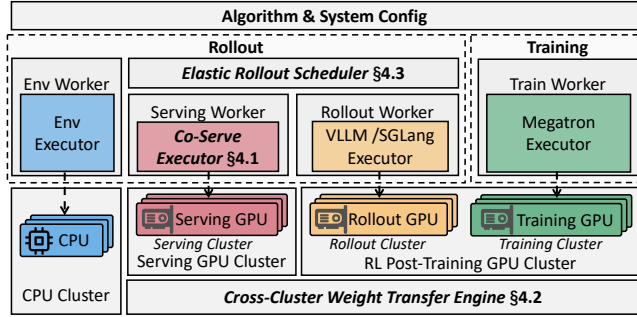


Figure 5. System Architecture of ROSE.

it can take up to 145 s and grow quickly with model size, becoming a bottleneck for frequent weight synchronization.

4 System Design

System Overview. To address the above challenges, we introduce *ROSE*, the architecture of which is illustrated in Figure 5. It includes three novel components: (1) an SLO-safe *co-serving executor* that co-locates *heterogeneous* serving and rollout LLMs on the same GPUs, dynamically sharing memory and compute while preserving serving SLOs (§4.1); (2) a *cross-cluster weight transfer engine* that asynchronously synchronizes model weights from the training cluster to the serving cluster with sharding and sparsity awareness (§4.2); and (3) an *elastic rollout scheduler* that dispatches rollouts across rollout and serving GPUs to realize the cooperative elasticity and achieve end-to-end training throughput improvement under varying serving and rollout load (§4.3).

Serving Cluster Setup. *ROSE* supports diverse serving deployments, including PD disaggregation vs. co-location and autoscaled vs. statically provisioned clusters. In our evaluation, we use PD disaggregation with a statically provisioned serving cluster and enforce P99 tail-latency SLOs on TTFT and TPOT. To avoid the initialization overhead of loading rollout models on demand, we pre-deploy commonly used rollout models alongside the heterogeneous serving LLMs in the serving cluster (e.g., by running `sglang.launch_server` or `vllm serve`) and expose endpoints for weight transfer and response generation. When there is no training job, rollout workers remain deactivated (i.e., not resident on the GPU) and consume at most 2 GB GPU memory, and they are activated during the rollout stage. We detail the concrete model pairing and cluster configuration in §6.

System Workflow. The user specifies the RL resource request N_{rl} and an upper bound on the number of serving GPUs that can be borrowed $N_{serving}$. Upon job submission, the RL cluster reserves N_{rl} GPUs, and the serving cluster selects up to $N_{serving}$ GPUs with the lowest recent KVC usage over a fixed window (e.g., 1 hour). To avoid contention among concurrent RL jobs, *ROSE* assigns each selected serving GPU to at most one RL job for rollouts. During this setup, *ROSE* initializes the RL runtime on the reserved RL GPUs and

launches rollout workers accordingly. On selected serving GPUs, *ROSE* activates the rollout runtime into GPU memory.

Each RL step follows a typical RL pipeline. First, *ROSE* initiates the rollout stage. Each trajectory involves multi-turn environment interactions, and the elastic rollout scheduler dispatches each turn to either dedicated rollout GPUs or borrowed serving GPUs to improve rollout throughput while enforcing serving SLOs. Second, on each serving GPU, the co-serving executor dynamically shares memory and compute between the heterogeneous serving and rollout LLMs, enforcing serving TTFT and TPOT SLOs while harvesting otherwise idle GPU cycles for rollouts. If serving load causes rollout stalls, the scheduler reroutes subsequent trajectories to underutilized GPUs. Third, the generated trajectories are fed into the training stage. Once training produces updated weights, *ROSE* invokes the cross-cluster weight transfer engine, which asynchronously pushes weight updates to serving GPUs while the next step’s intra-cluster synchronization and rollout execution proceed in parallel (§4.2).

4.1 SLO-Safe Co-Serving Executor

The co-serving executor runs rollouts opportunistically on serving GPUs while preserving serving TTFT and TPOT SLOs. This requires managing two sources of contention: *GPU memory* (dominated by KVC) and *GPU compute* (which directly affects token-level latency). Existing GPU multiplexing systems target either homogeneous models [41, 44] or workloads with comparable SLO requirements [7, 72, 82], and none simultaneously addresses the three challenges of cooperative elasticity: co-serving heterogeneous LLMs with incompatible KVC layouts, managing in-GPU prefix caching for inference workloads (Figure 1c), and yielding resources immediately when serving load spikes. Our evaluation (§6.5) confirms that naively adopting existing multiplexing engine [82] degrades rollout throughput while violating serving TTFT SLOs. We address these challenges through three techniques, enforcing two invariants: *serving-first memory* (serving always has priority on KVC capacity) and *serving-first compute* (serving always has priority on GPU compute).

VMM-based Cross-Model KV Cache Memory Sharing. Serving and rollout models are often heterogeneous (e.g., Qwen3-8B for serving vs. Qwen3-32B for rollouts) with incompatible KVC layouts due to different head dimensions, layer counts, and tensor parallelism degrees. Unlike PD-disaggregated inference [43, 88], which transfers KVC between hosts running the *same* model, co-serving requires two *heterogeneous* models to dynamically share a single GPU’s memory. Mainstream engines [28, 51] pre-allocate per-model KVC pools at initialization and cannot rebalance across different layouts—reallocating memory between models would require destroying one model’s KVC pool and reconstructing it with a different layout, incurring seconds-level overhead.

We leverage CUDA VMM to enable fast, flexible KVC rebalancing across heterogeneous models. We decouple *virtual*

KV address spaces from *physical* GPU pages: each model reserves a contiguous virtual KV address space that preserves its attention-kernel indexing, while all models share a global physical page allocator that maps and unmaps pages on demand. When serving load increases, we unmap physical pages from rollout's virtual address space and remap them into serving's virtual address space at page granularity (typically 2MB). This enables cross-model memory rebalancing without modifying KVC layouts or restarting attention kernels. We keep a lightweight runtime context warm on serving GPUs for fast rollout model (re-)activation. Activating Qwen3-32B completes within 5 s via PCIe/NVLink weight loading, avoiding the tens-of-seconds overhead of add-capacity elasticity.

Preemptive Memory Sharing Policy. Prefix caching is critical for rollout performance (Figure 1c), but existing multiplexing systems [72, 82] offload completed request KVC to CPU memory to free GPU capacity. However, rollout environment interactions are short-lived: consecutive interactions typically occur within 10 s [17], concentrating prefix reuse in a brief window. CPU-GPU transfer latency ($\sim$ ms) makes offloading ineffective for such short intervals. We instead keep rollout prefix cache resident in GPU to exploit short-term reuse, but this creates memory contention with serving that prior systems do not address. We note that prior KVC engines [6, 46] also use host memory offloading, which is ineffective for rollouts because weights are updated every RL step, quickly invalidating cached entries but increase high CPU memory pressure.

We address this tension through *preemptive memory sharing*: we keep rollout prefix cache in GPU under typical load, but aggressively reclaim it during serving bursts, relying on the short interaction window to recapture most reuse before entries expire. At the beginning of each RL step, the elastic rollout scheduler derives a per-GPU rollout KVC budget from the serving model's recent memory usage and reserves a fixed KVC headroom H (e.g., 20% of total GPU memory) for serving. This iteration-level budget stabilizes rollout memory usage under typical serving load, but it cannot react to sudden serving bursts.

The memory sharing policy proceeds in three steps. (1) *Burst trigger*: The co-serving executor continuously monitors serving KVC usage. When serving starts to consume the reserved headroom (i.e., serving KVC usage crosses a high-watermark within H), the executor enters a *pressure* state. (2) *Emergency cut*: In the pressure state, the executor immediately shrinks the rollout KVC budget by a fixed factor ($2\times$) and returns the freed physical pages to the shared allocator. It reclaims rollout KVC pages at request granularity, aborts the affected rollout requests, and notifies the rollout scheduler (§4.3) to reroute the affected trajectories to other underutilized GPUs. This one-time aggressive cut avoids repeated fine-grained reallocations and reduces allocation/reclamation churn during load bursts. (3) *Freeze*:

To prevent oscillation between reclaiming and regrowing rollout KVC under bursty traffic, the executor does not increase the rollout budget until the next RL step, when the rollout scheduler recomputes budgets using updated serving statistics. This conservative choice preserves serving SLOs, while the rollout scheduler absorbs the transient capacity loss by shifting subsequent rollout steps to underutilized GPUs, including dedicated rollout GPUs, as rollouts progress. We attach a short lease (e.g., 10 s) to each rollout KVC page and reclaim pages when the lease expires, bounding HBM consumption while capturing most prefix reuse within the typical environment interaction window.

Dual-SLO Admission Controller. Co-serving causes compute interference between rollout and serving under spatial co-location (analyzed in §6.2). Unlike serving requests that require millisecond-level TTFT (e.g., 100 ms) and TPOT (e.g., 50 ms) SLOs, rollouts tolerate seconds-level delays because preempted rollouts can be rerouted to dedicated rollout GPUs by the global scheduler (§4.3). We exploit this asymmetry through *temporal sharing* with serving-first admission control: only one workload executes on the GPU at a time while both reside in GPU memory, serving tokens are always prioritized, and rollout tokens are admitted only when sufficient SLO slack exists. When serving bursts arrive, we immediately yield compute—stalled rollouts simply migrate to underutilized GPUs without permanent progress loss.

We pre-profile runtime costs for both serving and rollout execution and use them to make online admission decisions. For prefill, we profile the latency of monolithic and chunked prefill as a function of prompt length, denoted by $\hat{T}_{\text{prf}}(L, m)$ where $m \in \{\text{mono}, \text{chunk}\}$. For decode, we profile the per-step latency as a function of batch size, denoted by $\hat{T}_{\text{dec}}(b)$. These profiles allow the executor to estimate, at each scheduling tick, the remaining slack under the serving TTFT and TPOT SLOs, and to admit rollout tokens only when sufficient slack is available. Although the serving stack uses PD disaggregation, we perform PD colocation for rollouts to maximize serving resource utilization. We further enable chunked prefill for rollouts (e.g., chunk size 512 tokens), which bounds the runtime of each rollout step and avoids head-of-line blocking for serving. Next, we describe the computation of SLO slack on prefiller and decoder instances.

On **prefiller** instances, we first compute the TTFT slack of pending serving requests at each scheduling tick. For a serving request r , let t_{now} be the current time and t_r^{arr} be its arrival time. Let B_{TTFT} be the configured TTFT SLO budget. Given prompt length L_r and the serving prefill mode m , the remaining TTFT slack is

$$S_r^{\text{prf}} = (t_r^{\text{arr}} + B_{\text{TTFT}}) - t_{\text{now}} - \hat{T}_{\text{prf}}(L_r, m). \quad (1)$$

We conservatively use the minimum slack among queued prefills as the TTFT slack for rollouts, $S_{\text{min}}^{\text{prf}} = \min_{r \in Q_{\text{prf}}} S_r^{\text{prf}}$.

On **decoder** instances, for a serving request r , let t_r^{last} be the time it produced its most recent token. Let B_{TPOT} be the configured TPOT budget, and let b be the current serving decode batch size. The remaining TPOT slack is

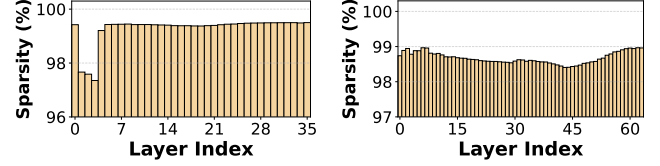
$$S_r^{\text{dec}} = \left(t_r^{\text{last}} + B_{\text{TPOT}} \right) - t_{\text{now}} - \hat{T}_{\text{dec}}(b). \quad (2)$$

We again use the minimum slack among active decodes, $S_{\min}^{\text{dec}} = \min_{r \in Q_{\text{dec}}} S_r^{\text{dec}}$. We admit rollout token generation only when two conditions hold. First, the serving workload has sufficient TTFT and TPOT slack to accommodate the additional compute for rollout tokens. Second, allocating the corresponding KVC pages for rollout does not reduce the serving model's available KVC capacity below a reserved headroom. If rollout prefill or decoding makes no progress for a fixed timeout (e.g., 2 seconds), the co-serving executor reports a stall to the global scheduler and drops this trajectory, allowing the scheduler to reroute it elsewhere.

4.2 Cross-Cluster Weight Transfer Engine

The serving and training clusters may reside in different datacenters connected by bandwidth-limited interconnects. As discussed in §3, cross-cluster communication overhead can become a bottleneck and undermine the benefits of cooperative elasticity. Existing communication systems [10, 33, 50, 67] exploit gradient sparsity for *intra*-cluster communication, but assume collective communication over fixed process groups with uniform sharding (typically data parallelism). Cooperative elasticity introduces two requirements absent in these settings: (1) dynamic GPU membership, as serving GPUs join or leave across RL steps, precludes fixed collectives; and (2) heterogeneous parallelism strategies between training and serving require automatic shard mapping. Additionally, we observe that RL post-training *weight deltas* are >95% sparse (Figure 6), a property uncommon in conventional LLM training that enables lossless compressed delta transfer for *cross*-cluster weight synchronization. We address these through three techniques.

Asynchronous Weight Transfer. Resource elasticity makes the set of participating serving GPUs *dynamic*: serving GPUs may join or leave between RL steps due to changing serving load. This precludes fixed collective groups and requires fault-tolerant, point-to-point transfer that gracefully handles membership changes. Following RollArt [17], we build the transfer engine on Mooncake Store as a relay layer that decouples training and serving: training workers push weights in fixed-size buckets (e.g., 64 MB) to relay workers asynchronously, and serving workers pull in larger batches (e.g., 1 GB) on demand without coordinating with training or other serving workers. This avoids establishing fixed communication groups and makes transfer robust to membership changes. We overlap cross-cluster transfer with NCCL-based intra-cluster synchronization so that rollout workers can resume without waiting for cross-cluster transfer to finish.



(a) FrozenLake-8B.

(b) ALFWORLD-32B.

Figure 6. Layer-wise sparsity ratio at 10th step.

Shard-aware Weight Transfer. Training and serving clusters adopt heterogeneous parallelism strategies (e.g., training with TP8×PP2 and serving with TP4), requiring automatic shard mapping across configurations. Naive approaches require manual resharding or full model aggregation before transfer. *ROSE* automatically infers each parameter's sharding rule by identifying the sharded dimension from the module type and parameter shape, computing per-rank slice ranges, and encoding this metadata in the Mooncake object key. On the training side, each device pushes its local shard asynchronously rather than first all-gathering the full model. To avoid redundant sends, each data-parallel rank transmits a mutually exclusive set of shards, parallelizing transfers and improving link utilization. On the serving side, each rank deterministically derives which buckets to pull based on the encoded metadata, fetching only its needed shards rather than a full model replica. This substantially reduces transfer volume and makes weight transfer transparent to the RL algorithm. We support tensor parallelism (TP) and pipeline parallelism (PP).

Sparsity-aware Weight Transfer. Transfer time scales with data volume, so naively synchronizing full weights becomes prohibitive as model size grows. Prior communication systems [10, 67] exploit gradient sparsity to improve *intra*-cluster communication efficiency, but lossless gradient sparsity is rare in LLM training [40, 49, 76]. We observe that RL post-training produces a natural source of lossless sparsity: the *weight differentials* $\Delta W_t = W_t - W_{t-1}$ between consecutive steps are highly sparse, because RL algorithms employ gradient-stabilization techniques (e.g., reference models, KL penalties, and conservative update rules) [53, 81] that constrain policy drift. We exploit this weight-differential sparsity for *cross*-cluster transfer after training completes.

We define *sparsity ratio* as the fraction of zero elements in ΔW_t . Figure 6 reports the layer-wise sparsity for Qwen3-8B and Qwen3-32B at the 10th RL step. Across layers, more than 95% of elements in ΔW_t are zero, consistent with prior observations in RL post-training [29, 62]. §6.3 confirms that this high sparsity persists across RL steps. The engine therefore ships sparse deltas rather than full replicas.

To exploit this sparsity efficiently, we compress ΔW_t in COO (coordinate) format for transfer and keep the previous-step weights W_{t-1} resident on local devices. At each update, we reconstruct the current weights by applying ΔW_t to W_{t-1} . This additive update introduces at most ~1 s of compute

overhead, which is small relative to the cross-cluster transfer time (up to minutes). To further reduce overhead, we shard the COO tensors according to the chosen parallelism strategy so that each device applies only its local shard, avoiding sparse-to-dense materialization and redundant additions.

4.3 Elastic Rollout Scheduler

The rollout scheduler harnesses cooperative elasticity to orchestrate trajectory generation across rollout and serving GPUs. However, rollout workers on serving GPUs expose a response generation endpoint, and their data plane differs from that of rollout workers running on dedicated rollout GPUs. We therefore introduce a unified rollout proxy to allow the scheduler to manage heterogeneous rollout workers sharing a common interface for response generation and per-trajectory metric collection. Next, we describe how the scheduler decides how many trajectories to offload to serving GPUs and how it places each trajectory.

Turn-wise Concurrency-aware Routing. The GPU compute and memory jointly bound the number of trajectories that can execute efficiently on the dedicated rollout GPUs. We perform offline profile and cap rollout concurrency at a workload-dependent threshold (e.g., 16 per GPU for Qwen3 models with 32K context length) to avoid KVC pressure and excessive scheduling overhead. When the instantaneous rollout demand exceeds this limit, the scheduler offloads the excess trajectories to available serving GPUs. Since multi-turn agentic RL alternates environment interaction with LLM generation, *ROSE* schedules rollouts at *turn* granularity rather than pinning each trajectory to a single GPU. This fine-grained control allows subsequent turns of a trajectory to spill over to serving GPUs under contention, and to migrate back to rollout GPUs once capacity becomes available.

Cache-Affinity Placement. To maximize the benefit of prefix caching, the rollout scheduler employs a cache-affinity placement policy across both dedicated rollout GPUs and opportunistic serving GPUs. For each trajectory, the scheduler records the rollout worker that served its previous turn, which typically retains the trajectory’s prefix KVC. For each new turn, the scheduler first routes the request to the cache-affine worker if it has available capacity on a rollout GPU or, for a serving GPU, if admitting the request would not violate serving SLOs. If the cache-affine worker is unavailable, the scheduler falls back to a load-aware policy by dispatching the request to the least-loaded rollout GPU when possible; otherwise, it dispatches to the least-loaded eligible serving GPU. If neither pool has capacity, the request is queued until resources become available.

Fault Tolerance and Recovery. The rollout scheduler uses a heartbeat mechanism to monitor the liveness of each serving worker. When it receives an execution stall signal from the co-serving executor (§4.1) or detects a failure via health checks, the scheduler promptly reroutes the affected trajectories to other available GPUs, enabling fast recovery.

Extensions to Other Deployment Settings. *ROSE* can generalize to two other LLM serving deployments. *PD colocation.* When prefill and decode are co-located on the same serving instance, the executor derives per-tick compute slack for rollout admission as the minimum one between TTFT and TPOT slack. *Autoscaling.* This can leave a fraction of GPUs idle under low traffic. *ROSE* can run rollouts on these idle GPUs and leverage the fast model swap mechanism in §4.1 to utilize GPU compute and memory without contending with serving traffic. Due to the cost of large-scale evaluation across multiple production configurations, we focus on the mainstream PD-disaggregated deployment used in industry.

5 Implementation

We implement ~5k lines of Python atop agentic training framework ROLL [64] and serving framework vLLM [28].

Agentic RL training. The rollout scheduler and the push side of the weight transfer engine are implemented inside ROLL. *ROSE* uses Megatron-LM [57] for training, vLLM for rollout/serving with request migration, and ROLL’s native environment runtime to manage environments.

Serving engine. The pull side of the transfer engine and the co-serving executor are built atop vLLM 0.10.0 [28]. We also implement a Ray-based [39] load-balancing scheduling policy to route serving requests.

Relay worker. We use Mooncake v0.3.8 in the relay worker, allowing ROLL to publish updated weights and vLLM to pull them. We extend Mooncake with shard awareness and sparsity awareness to reduce communication overhead.

Environment runtime. Environments run in CPU-only containers on a separate Kubernetes cluster, communicating with rollout workers via K8S API calls. This isolates environment execution from GPU workloads. *ROSE*’s design is orthogonal to environment placement and extends to GPU-accelerated environments.

6 Performance Evaluation

Models and Training Configurations. We use Qwen3-8B/32k for FrozenLake [9] and Qwen3-32B/32k for ALF-World [58]. Both are widely adopted benchmarks in the agentic RL literature [12, 17, 64]. We train them with GRPO [53] and DAPO [81]. For all tasks, we set the group size to 16 and use batch sizes of 256 and 1024 for Qwen3-8B and Qwen3-32B, respectively. For DAPO, the rollout stage continues until it collects the target number of trajectory groups with non-zero reward variance. We adjust the maximum number of concurrent trajectories at each step based on the previous step to speed up the collection of valid trajectories. We train the 8B and 32B models on 16 and 48 dedicated GPUs, with (Rollout, Training) allocations of (8, 8) and (16, 32). Rollout TP is 1 (8B) and 4 (32B), while training parallelism (TP, PP, CP) is (4, 1, 1) and (8, 1, 2). We cap borrowed serving GPUs at 16 (8B) and 64 (32B), and set the per-device rollout batch

size to 16 to avoid contention (see Appendix A) based on profiling.

Cluster Setup. We run agentic RL training on an H800 cluster with up to 48 GPUs and online serving on a separate H800 cluster with 64 GPUs. Within each cluster, nodes are connected via 400 Gbps InfiniBand. Due to operational constraints, our evaluation uses a 200 Gbps Ethernet link for cross-cluster communication. We further evaluate communication efficiency under different link bandwidths in §6.3. We use NCCL and Mooncake Store [46] for intra- and cross-cluster weight transfer. For the serving cluster, we pre-deploy Qwen3-8B and Qwen3-32B as rollout models, which are used by 40% and 19% of agentic RL training jobs in our cluster, respectively, and together cover most workloads. We co-locate them with similarly sized heterogeneous serving models (Qwen3-8B with Qwen2.5-7B, Qwen3-32B with Qwen2.5-32B) because they use compatible parallelism configurations. We set the prefill-to-decoding instance ratio to 1:3. We replay the 24-hour online serving trace from Microsoft [60], and run a load-balancing policy to route online serving requests.

Baselines and Training Recipe. We evaluate *ROSE* against state-of-the-art RL post-training systems. *ROSE* is agnostic to the choice of on-policy or off-policy RL algorithms, and also supports fully asynchronous training (e.g., AReaL [13], evaluated in §6.5). Due to the substantial overhead of agentic RL, we adopt one-step off-policy training [37] for all approaches. We use ROLL [64] as our resource-fixed baseline, as it provides better support for agentic tasks than verL [56]. We follow the design of existing commercial products [42, 62] to implement a ServerlessRL baseline, where the number of available serverless GPUs is set to the maximum number of available spot GPUs, and we configure the function timeout to 15 minutes [1]. On *ROSE* and ROLL, we perform training until the models converge. Specifically, for GRPO, we train Qwen3-8B and Qwen3-32B for 100 and 40 steps, respectively, while for DAPO, we use 50 and 25 steps. For RLBoost, to capture the impact of spot GPU availability, we reproduce a representative 2-hour window characterized by high resource volatility from the traces (see Appendix B). We run RLBoost within this time window using GRPO.

Metrics. We measure training throughput as the total number of input and output tokens processed per global step divided by the step time [23, 89]. We use the average critic score as the accuracy metric. For serving, we set P99 SLOs for (TPOT, TTFT) to (150 ms, 500 ms) for Qwen2.5-7B and (450 ms, 1000 ms) for Qwen2.5-32B [83, 88].

6.1 End-to-End Evaluation

Model Convergence. We report the critic scores over the training in Figure 8. It shows that *ROSE* preserves the model convergence due to its algorithm-agnostic design, achieving final scores nearly identical to the baselines.

End-to-End Throughput. Figure 7 compares the end-to-end throughput across tasks. Compared with ROLL, *ROSE*

achieves average throughput improvements of 1.31× and 1.46× with GRPO, the maximum throughput increase is 2.16× and 1.76×. The advantage of *ROSE* becomes more pronounced under the DAPO algorithm, where average throughput increases by 1.42× and 3.31× with a maximum of 4.82×. We attribute the throughput gains to three components. (1) *Cooperative elasticity via co-serving executor*: by harvesting idle serving GPUs, *ROSE* effectively expands the rollout GPU pool. With 16 additional serving GPUs, rollout time decreases by 1.69× (Figure 9b), confirming that the co-serving executor successfully converts serving slack into rollout capacity. (2) *Elastic rollout scheduler*: DAPO’s redundant sampling can launch up to 5.7× the base batch size (Figure 1d). The resource-fixed baseline cannot absorb this burst and suffers severe contention, whereas *ROSE*’s scheduler dynamically offloads excess trajectories to serving GPUs. The scheduler’s turn-wise routing and KVC affinity together improve rollout efficiency by up to 1.48× (Table 3). (3) *Weight transfer engine*: cross-cluster synchronization completes within 21 s for Qwen3-32B (§6.3), preventing weight updates from becoming a bottleneck between steps. Moreover, *ROSE* eliminates the allocation overhead that plagues elastic baselines (0% vs. 6.8–26.1% for RLBoost and λ RL, Table 1). Across all experiments, *ROSE* keeps the serving P99 latency within target SLOs (§6.2). We further compare with AReaL [13], a fully asynchronous RL baseline, in §6.5.

Comparison with Elastic Baselines. Figure 9a compares rollout efficiency against λ RL and RLBoost. λ RL uses 16 and 32 serverless GPUs for rollouts of the 8B and 32B models, respectively, which is consistent with the maximum number of GPUs available to RLBoost. We observe that allocating more GPUs to rollouts yields up to 1.31× and 1.23× speedups over ROLL for Qwen3-8B and Qwen3-32B, respectively. Although RLBoost experiences fluctuating spot GPU availability, its resource allocation changes less frequently than that of λ RL, which reallocates resources at a fixed 15-minute lease interval. As a result, RLBoost achieves 1.41× and 1.48× speedups over ROLL for Qwen3-8B and Qwen3-32B, respectively, although its gains are still constrained by the instability of spot GPUs. *ROSE* further reduces rollout time by 1.20× and 1.26× compared to RLBoost. This improvement suggests that opportunistically leveraging serving GPUs provides more abundant and stable compute for rollouts while avoiding frequent preemption, thereby shortening rollout time.

Model	λ RL	RLBoost	<i>ROSE</i>
Qwen3-8B	16.1%	7.3%	0%
Qwen3-32B	26.1%	6.8%	0%

Table 1. Ratio of allocation overhead to training time.

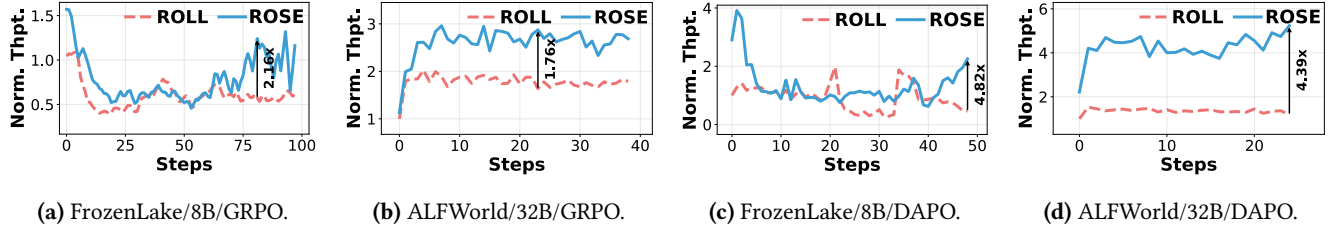


Figure 7. ROSE's end-to-end throughput improvements. The data are normalized to the baseline's first step.

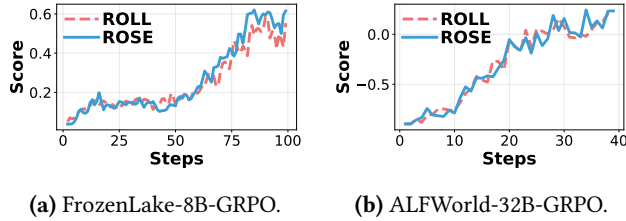


Figure 8. End-to-end critic scores for (a) 8B and (b) 32B models using the GRPO algorithm.

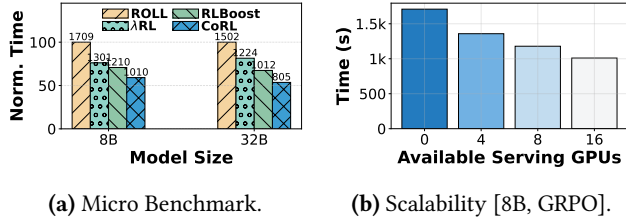


Figure 9. End-to-end evaluation. (a) Rollout time compared with baselines. (b) Scalability of ROSE on Qwen3-8B with GRPO as Serving GPUs increase.

Allocation Overhead. We further analyze the allocation overhead of elastic resource management schemes. We quantify the total *preempted GPU time* as the product of the number of preempted GPUs and the per-preemption overhead, and normalize it by the total GPU time. As shown in Table 1, λ RL incurs the highest overhead (up to 26.1%), because serverless GPUs are leased for fixed durations that are often shorter than a complete rollout, resulting in frequent preemptions and restarts. RLBoost experiences less frequent preemptions on spot instances, but still incurs non-negligible overhead (6.8–7.3%). In contrast, ROSE eliminates nearly all allocation overhead, thereby introducing the smallest training-time overhead.

Serving GPU Availability. The amount of underutilized capacity in the serving cluster also affects rollout performance. To quantify this effect, we measure the average rollout time of Qwen3-8B on FrozenLake using eight dedicated rollout GPUs over the first five RL steps, while varying the number of available serving GPUs. As shown in Figure 9b, rollout time decreases as the serving GPU quota increases. With an additional 16, 8 and 4 serving GPUs, ROSE reduces rollout time by 1.69 $\times$, 1.45 $\times$, and 1.26 $\times$, respectively. These results show that ROSE can effectively harvest underutilized serving resources to reduce the rollout overhead.

Table 2. [Co-Serve Executor]. The impact of memory sharing policy for rollout and SLO efficiency.

Model	Policy	Rollout	TTFT (ms)		TPOT (ms)	
		Time (s)	P95	P99	P95	P99
Qwen3-8B	Static Partition	776.7	299.5	330.95	1462.5	1488.2
	+ Memory Preemption	591.7	348.8	517.9	153.6	162.8
	+ Prefix Caching	469.3	318.5	333.6	173.2	186.3
Qwen3-32B	Static Partition	1310.5	595.6	603	6835.9	7021.1
	+ Memory Preemption	959.7	596.7	651.3	493.5	503.8
	+ Prefix Caching	920.4	607.2	640.6	477.9	483.2

6.2 Analysis of Co-serving Executor

To analyze the efficiency of the co-serving executor, we use the end-to-end setups for Qwen3-8B and Qwen3-32B and run GRPO for five steps. Because evaluation incurs substantial overhead, we use the checkpoint at step 50, which yields moderate rollout time and fluctuating serving load.

Preemptive Dynamic Memory Sharing Policy. This policy incorporates two memory-sharing optimizations, namely *memory preemption* and *prefix caching*. We run rollouts on serving GPUs via spatial GPU sharing and measure end-to-end rollout time, along with serving P95 and P99 TTFT and TPOT, to quantify rollout efficiency and SLO compliance. We first evaluate memory preemption. As a *static partition* baseline, we split GPU memory evenly between serving and rollout LLMs. Compared to *static partitioning*, memory preemption reduces P99 TPOT latency by 9.14 $\times$ for Qwen3-8B and 13.94 $\times$ for Qwen3-32B, indicating that it absorbs bursty serving-side memory demand and reduces tail latency.

Second, we enable *prefix caching* atop memory preemption. This reduces rollout time by 1.26 $\times$ for Qwen3-8B and 1.04 $\times$ for Qwen3-32B, with only a slight increase in serving tail latency. These results indicate that prefix caching mainly improves rollout throughput, but does not by itself ensure serving SLO compliance. Overall, the memory sharing policy effectively limits tail-latency inflation, but without explicit SLO-aware scheduling, P99 latency still misses our target.

Dual-SLO Admission Controller. This controller compares the current serving TTFT and TPOT against their SLO targets and time-multiplexes between serving traffic and rollout requests to maintain SLO compliance. We evaluate three policies under the same setup in Figure 11: *TTFT-only*, *TPOT-only*, and *Dual-SLO*. *TTFT-only* enforces only the TTFT SLO, while *TPOT-only* enforces only the TPOT SLO. The average step time is similar across all three policies, suggesting that the policy choice has limited impact on rollout time.

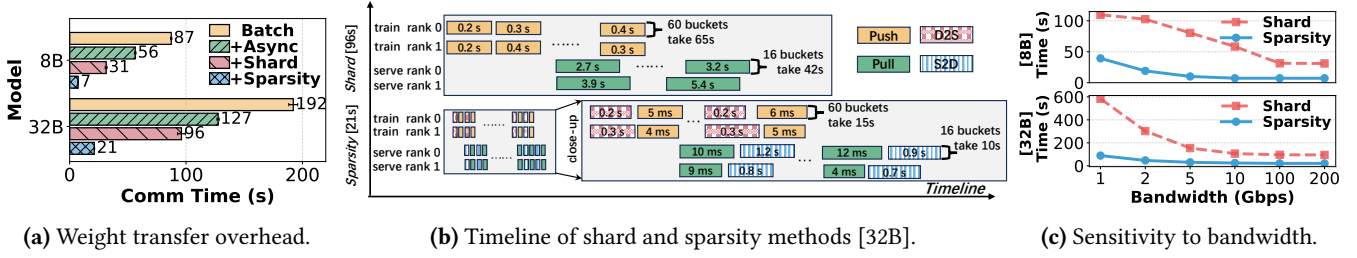


Figure 10. [Transfer Engine] (a) Cross-cluster weight transfer time under different optimizations; each optimization is additive over the previous one. (b) Timeline breakdown of shard-aware and sparsity-aware transfer for Qwen3-32B. D2S denotes the dense-to-sparse conversion, and S2D denotes the sparse-to-dense conversion. (c) Sensitivity of shard-aware and sparsity-aware transfer of different LLMs to cross-cluster bandwidth.

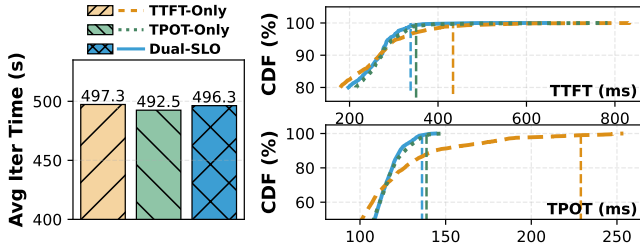


Figure 11. Analysis of Dual-SLO Admission Controller.

Compared to the single-objective baselines, *Dual-SLO* consistently achieves lower P99 tail latency on both metrics. It reduces P99 TPOT by 3.4% and 76.3% relative to *TPOT-only* and *TTFT-only*, respectively, and reduces P99 TTFT by 3.7% and 28.4% over the same baselines. Overall, *Dual-SLO* explicitly provides guarantees for both TTFT and TPOT while preserving rollout efficiency. We further explore the sensitivity of the co-serving executor to online serving load (Appendix D) and prefix caching lease time (Appendix C), demonstrating its robustness.

6.3 Effectiveness of Transfer Engine

We evaluate the transfer engine using 16 GPUs for training and 16 GPUs for serving, and measure cross-cluster communication overhead over three RL steps. Unless otherwise specified, the training and serving clusters are connected by a 200 Gbps Ethernet link.

Analysis of Weight Transfer Optimizations. Figure 10a quantifies the cross-cluster communication efficiency of *asynchrony*, *shard-awareness*, and *sparsity-awareness* for Qwen3-8B and Qwen3-32B. As a baseline (*batch*), training workers all-gather model weights and transmit the entire model, and then serving workers pull model weights from relay workers into GPU memory. First, *asynchronous* transfer streams parameters at bucket granularity and pipelines publishing with pulling. Compared to *batch*, it reduces end-to-end communication time by 1.6× (Qwen3-8B) and 1.5× (Qwen3-32B), primarily by overlapping the sender-side publish with the receiver-side pull rather than reducing transmitted bytes.

Second, we enable *shard-awareness*: each training worker publishes only its local shard, and each serving worker pulls

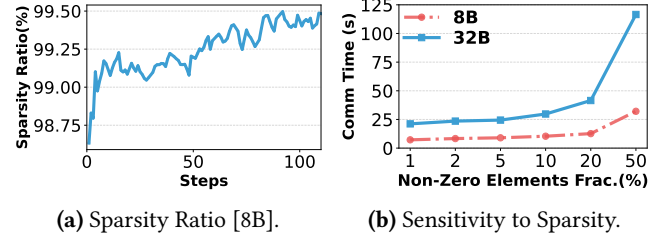


Figure 12. [Analysis of Sparsity]. (a) The sparsity of weight differentials across steps for Qwen3-8B. (b) The sensitivity of transfer engine to sparsity.

only the shards it hosts. This further reduces communication time by 1.8× (Qwen3-8B) and 1.3× (Qwen3-32B). Moreover, Figure 10b (top) illustrates the Qwen3-32B timeline. On the sender side, each training worker streams ~60 buckets (64 MB each); each bucket takes 0.2–0.4 s to push, for a total of 65 seconds. On the receiver side, serving workers pull the corresponding weight buckets from the relay and load them into GPU memory, taking 42 s in total. Overall, shard-awareness prevents redundant weight all-gather and avoids pulling and re-sharding a full replica on each serving worker, while increasing effective parallelism by utilizing multiple cross-cluster links.

Third, *sparsity-awareness* provides the largest gain, reducing communication time by 4.4× (Qwen3-8B) and 4.6× (Qwen3-32B). Figure 10b (bottom) shows the 32B timeline. Because weight deltas are highly sparse, each bucket is much smaller: per-bucket push and pull latency drops from hundreds of milliseconds to just a few milliseconds. The remaining per-bucket overhead is dominated by (de)sparsification, specifically Dense-to-Sparse (D2S) on the training side and Sparse-to-Dense (S2D) on the serving side, which stays sub-second. As a result, end-to-end transfer time decreases to 21 s. Overall, our optimized cross-cluster weight transfer engine reduces end-to-end communication time by 12.4× for Qwen3-8B and 9.1× for Qwen3-32B, bringing cross-cluster transfer down to tens of seconds and close to intra-cluster weight transfer overhead.

Sensitivity to Cross-Cluster Bandwidth. We study the sensitivity of shard-aware and sparsity-aware transfer to

Table 3. [Rollout Scheduler]. The speedup of the elastic rollout scheduler on rollout time.

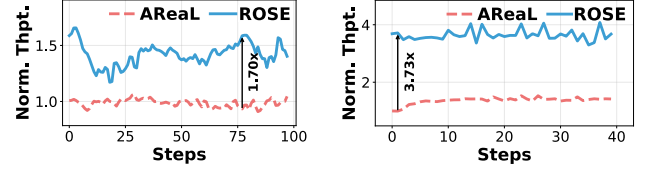
Policy	Qwen3-8B	Qwen3-32B
Baseline	1.00×	1.00×
+ Turn-Wise Routing	1.11×	1.08×
+ KVC Affinity	1.16×	1.48×

Table 4. ROSE vs. alternative serving engines: rollout time and serving SLO metrics.

Model	Method	RolloutTTFT- Time (s)	TPOT- P99 (ms)	TPOT- P99 (ms)
8B	ROSE	496.3	338.1	136.1
	ServerlessLLM	–	314.8	117.8
	ServerlessLLM+Rollout	651.7	1166.1	135.6
	Prism	731.7	973.2	115.4
32B	ROSE	960.1	837.5	398.1
	ServerlessLLM	–	716.1	312.8
	ServerlessLLM+Rollout	1161.8	2426.2	565.3
	Prism	1301.2	1625.4	351.7

cross-cluster bandwidth by throttling link capacity from 200 Gbps to 1 Gbps using `tc` command, and measuring end-to-end communication overhead for Qwen3-8B (Figure 10c, top) and Qwen3-32B (Figure 10c, bottom). For Qwen3-8B, shard-aware transfer increases from 31 s (200 Gbps) to 109 s (1 Gbps), while sparsity-aware transfer stays within 7–39 s, delivering a 2.8×–4.4× speedup. For Qwen3-32B, shard-aware transfer grows from 96 s to 584 s, whereas sparsity-aware transfer remains within 21–89 s (4.6×–6.6× faster). Overall, sparsity-awareness flattens the scaling curve by reducing transferred bytes. Even at 5 Gbps, transfer completes within 10 s (Qwen3-8B) and 30 s (Qwen3-32B). Because cross-cluster transfer is overlapped with training and intra-cluster communication, it increases end-to-end training time by at most 5%.

Analysis of Weight Differential Sparsity. Figure 12a shows that the weight-differential sparsity for Qwen3-8B stays around 99% throughout training. This indicates that sparsity is persistent rather than a transient artifact of a particular training phase. We next vary the non-zero fraction in Figure 12b for Qwen3-8B and Qwen3-32B to evaluate the sensitivity of our sparsity-aware transfer engine to the sparsity ratio. As the non-zero fraction increases, communication overhead rises and the benefit of sparse transfer diminishes. Beyond ~20%, sparse-format metadata (e.g., indices) and (de)sparsification overhead begin to offset the reduction in transmitted weights. In our workloads, the measured non-zero fraction remains well below this threshold, enabling consistently efficient weight propagation.



(a) FrozenLake-8B-AReal.

(b) ALFWorld-32B-AReal.

Figure 13. ROSE under fully asynchronous RL training workloads. We monitor the average throughput between consecutive RL steps.

6.4 Effectiveness of Rollout Scheduler.

We follow the end-to-end setups and evaluate the elastic rollout scheduler using Qwen3-8B and Qwen3-32B with GRPO algorithm for the first five RL steps. Table 3 analyzes the contribution of two heuristics adopted by the rollout scheduler. As a baseline, we pin each trajectory to a fixed rollout worker for its entire lifetime. This static assignment leads to load imbalance because trajectory runtimes vary widely. As a result, enabling turn-wise routing reduces rollout time by 1.11× (Qwen3-8B) and 1.08× (Qwen3-32B), demonstrating the benefit of fine-grained, flexible routing. Adding KVC affinity yields further improvements, bringing the cumulative speedup to 1.16× for Qwen3-8B and 1.48× for Qwen3-32B. This is because larger models incur heavier prefill costs, making KV reuse more effective at reducing per-turn overhead. Overall, these simple heuristics substantially improve rollout efficiency. The scheduler adds negligible overhead (at most 10 ms per decision) throughout training, suggesting that it scales well.

6.5 Extension to Other Scenarios

We evaluate ROSE under alternative serving engines and RL algorithms to demonstrate generality.

Bidirectional Autoscaling as Serving Engine. We replace ROSE’s co-serving executor with ServerlessLLM [14], a state-of-the-art open-source LLM autoscaling engine, to orchestrate serving GPUs via bidirectional autoscaling. We follow the end-to-end setup (§6.2) and run GRPO for ten RL steps. We configure ServerlessLLM to preserve serving TTFT and TPOT P99 SLOs. Table 4 reports the results. Without rollout co-location, ServerlessLLM achieves lower serving latency than ROSE (e.g., TTFT-P99: 314.8 ms vs. 338.1 ms for 8B) because it does not share GPU resources. However, once rollout is enabled (ServerlessLLM + Rollout), the bidirectional autoscaling must repeatedly evict and reload models when serving load spikes, inflating rollout time by 1.31× (8B) and 1.21× (32B) compared to ROSE, while severely violating TTFT SLOs (1166 ms and 2426 ms). This confirms our analysis in §3.2: bidirectional autoscaling cannot safely harvest serving slack under bursty traffic.

Prism as Serving Engine. We evaluate Prism [82], a GPU multiplexing system for heterogeneous LLM serving workloads. We configure Prism to co-schedule rollout and serving

requests, setting rollout SLOs to 4× the serving SLOs, and run GRPO for ten RL steps. Unlike *ROSE*, Prism does not support fast memory preemption or prefix caching for rollouts. When request SLOs cannot be met, Prism defers its execution, which significantly degrades both rollout throughput and serving TTFT. As shown in Table 4, Prism inflates rollout time by $1.47\times$ (8B) and $1.36\times$ (32B) compared to *ROSE*, while also violating TTFT SLOs (973 ms and 1625 ms vs. targets of 500 ms and 1000 ms). This confirms that generic GPU multiplexing without RL-aware co-serving cannot effectively handle rollouts and serving under bursty traffic.

Comparison to AReaL. To compare *ROSE* against fully asynchronous RL training, we integrate AReaL [13], a fully asynchronous off-policy RL system that keeps GPUs saturated by continuously generating trajectories without waiting for training to complete. We follow the end-to-end setup (§6) and compare throughput under GRPO in Figure 13. *ROSE* achieves $1.44\times$ (Qwen3-8B) and $2.69\times$ (Qwen3-32B) throughput compared with AReaL. Although AReaL eliminates GPU idle time via full asynchrony, it is constrained by its fixed GPU budget and may introduce stale trajectories that reduce sample efficiency. *ROSE* complements this by expanding effective GPU capacity through cooperative elasticity, providing throughput gains orthogonal to asynchronous execution.

7 Related Works

Agentic RL Training Systems. Many systems optimize conventional single-turn RL post-training [5, 16, 19, 22–24, 45, 55, 70, 79, 89, 90]. The rise of agentic LLMs has also motivated agentic RL training frameworks [4, 12, 17, 58, 64, 69]. However, these agentic systems typically assume fixed resources. A few elastic RL systems exploit spot instances [70] or serverless GPUs [42, 62]. *ROSE* explores underutilized serving GPUs for resource elasticity and is complementary to these elastic approaches.

Serving GPU Sharing. GPU multiplexing has been widely studied for years. Prior DL systems [15, 18, 32, 73, 86] interleave multiple models on the same GPUs to improve utilization. More recently, LLM serving systems multiplex GPUs across multiple LLM workloads [7, 74, 82, 85]. However, they do not target co-serving RL rollouts with online LLM serving.

Sparsity-based Optimization. Recent systems including Check-N-Run [8] and LowDiff [78] leverage sparsity in the *weight differential* to reduce storage and checkpoint overhead. Many training systems [2, 10, 33, 67] exploit gradient sparsity to cut communication cost. Inspired by these works, we observe the sparsity in the weight differential of RL training and leverage it to reduce the communication overhead.

Cycle Stealing. Harvesting idle resources across workloads is a well-studied concept. Cycle stealing [21] demonstrated that beneficiaries gain unbounded benefit from donors' idle cycles with only slight donor penalty. In the GPU era, Ekya [3]

balances inference and continuous retraining on edge GPUs, and Lyra [30] loans idle serving GPUs to training jobs. *ROSE* extends this philosophy to co-serving heterogeneous LLMs, additionally handling incompatible KVC layouts, prefix caching contention, and cross-cluster weight synchronization.

8 Conclusion

In this paper, we present *ROSE*, a cooperative, elastic post-training system that opportunistically harvests serving GPUs to accelerate agentic RL training. *ROSE* combines (i) a co-serving executor for SLO-safe compute and memory sharing, (ii) a cross-cluster weight transfer engine for low-overhead weight synchronization, and (iii) an elastic rollout scheduler that efficiently realizes the cooperative elasticity. Extensive experiments show that *ROSE* improves training efficiency over baselines while preserving serving SLOs.

References

- [1] Alibaba Cloud. 2026. Creating a GPU function. <https://www.alibabacloud.com/help/en/functioncompute/fc/user-guide/creating-a-gpu-function/>. (2026). Accessed: 2026-04.
- [2] Dan Alistarh, Demjan Grubic, Jerry Z. Li, Ryota Tomioka, and Milan Vojnovic. 2017. QSGD: communication-efficient SGD via gradient quantization and encoding. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 1707–1718.
- [3] Romil Bhardwaj, Zhengxu Xia, Ganesh Ananthanarayanan, Junchen Jiang, Yuanhao Shu, Nikolaos Karianakis, Kevin Hsieh, Paramvir Bahl, and Ion Stoica. 2022. Ekya: Continuous Learning of Video Analytics Models on Edge Compute Servers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 119–135. <https://www.usenix.org/conference/nsdi22/presentation/bhardwaj>
- [4] Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R. Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. SkyRL-Agent: Efficient RL Training for Multi-turn LLM Agent. *arXiv preprint arXiv:2511.16108* (2025).
- [5] Rongxin Cheng, Kai Zhou, Xingda Wei, Siyuan Liu, Mingcong Han, Mingjing Ai, Yeju Zhou, Baoquan Zhong, Wencong Xiao, Rong Chen, and Haibo Chen. 2025. Fast LLM Post-training via Decoupled and Best-of-N Speculation. *arXiv preprint arXiv:2511.16193* (2025).
- [6] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).
- [7] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving. In *ICML*.
- [8] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. 2022. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 929–943.
- [9] Farama Foundation. 2024. Gymnasium - FrozenLake Environment. https://gymnasium.farama.org/environments/toy_text/frozen_lake/. (2024). Accessed: 2025-09.
- [10] Jiawei Fei, Chen-Yu Ho, Atal N Sahu, Marco Canini, and Amedeo Sapia. 2021. Efficient sparse collective communication and its application to

- accelerate distributed deep learning. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 676–691.
- [11] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. Group-in-Group Policy Optimization for LLM Agent Training. *arXiv preprint arXiv:2505.10978* (2025).
 - [12] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. Group-in-Group Policy Optimization for LLM Agent Training. *arXiv preprint arXiv:2505.10978* (2025).
 - [13] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *arXiv preprint arXiv:2505.10978* (2025).
 - [14] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *OSDI'24*.
 - [15] Wei Gao, Zhuoyuan Ouyang, Peng Sun, Tianwei Zhang, and Yonggang Wen. 2025. IceFrog: A Layer-Elastic Scheduling System for Deep Learning Training in GPU Clusters. *IEEE Transactions on Parallel and Distributed Systems* 36, 6 (2025), 1071–1086. <https://doi.org/10.1109/TPDS.2025.3553137>
 - [16] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2025. RollPacker: Mitigating Long-Tail Rollouts for Fast, Synchronous RL Post-Training. *arXiv preprint arXiv:2509.21009* (2025).
 - [17] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, Ju Huang, Siran Yang, Yongbin Li, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2025. RollArt: Scaling Agentic RL Training via Disaggregated Infrastructure. *arXiv preprint arXiv:2512.22560* (2025).
 - [18] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale preemption for concurrent {GPU-accelerated}{DNN} inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 539–558.
 - [19] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, Huazhong Ji, Wenjie Liu, Yu Huang, Yixiang Zhang, Chenyi Pan, Jing Wang, Xin Huang, Chunsheng Li, and Jianping Wu. 2025. AsyncFlow: An Asynchronous Streaming RL Framework for Efficient LLM Post-Training. *arXiv preprint arXiv:2507.01663* (2025).
 - [20] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Yicheng Chen, Cunyin Peng, Jinjie GU, and Chenyi Zhuang. 2025. Exploring Superior Function Calls via Reinforcement Learning. *arXiv preprint arXiv:2508.05118* (2025).
 - [21] Mor Harchol-Balter, Cuihong Li, Takayuki Osogami, Alan Scheller-Wolf, and Mark S. Squillante. 2003. Cycle stealing under immediate dispatch task assignment. In *Proceedings of the Fifteenth Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '03)*. Association for Computing Machinery, New York, NY, USA, 274–285. <https://doi.org/10.1145/777412.777462>
 - [22] Eric Harper, Somsubhra Majumdar, Oleksii Kuchaiev, Li Jason, Yang Zhang, Evelina Bakhturina, Vahid Noroozi, Sandeep Subramanian, Koluguri Nithin, Huang Jocelyn, Fei Jia, Jagadeesh Balam, Xuesong Yang, Micha Livne, Yi Dong, Sean Naren, and Boris Ginsburg. 2025. NeMo: a toolkit for Conversational AI and Large Language Models. (2025). <https://github.com/NVIDIA/NeMo>
 - [23] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2025. History Rhymes: Accelerating LLM Reinforcement Learning with RhymeRL. *arXiv preprint arXiv:2508.18588* (2025).
 - [24] Jian Hu, Xibin Wu, Weixun Wang, Dehao Zhang, Yu Cao, et al. 2024. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. *arXiv preprint arXiv:2405.11143* (2024).
 - [25] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: scaling large language model training to more than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI'24)*. USENIX Association, USA, Article 41, 16 pages.
 - [26] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770* (2024).
 - [27] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
 - [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
 - [29] Jung Hyun Lee, Jeonghoon Kim, June Yong Yang, Se Jung Kwon, Eunho Yang, Kang Min Yoo, and Dongsoo Lee. 2025. LRQ: Optimizing Post-Training Quantization for Large Language Models by Learning Low-Rank Weight-Scaling Matrices. *arXiv preprint arXiv:2407.11534* (2025).
 - [30] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. 2023. Lyra: Elastic Scheduling for Deep Learning Clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 835–850. <https://doi.org/10.1145/3552326.3587445>
 - [31] Zhiwei Li, Yong Hu, and Wenqing Wang. 2025. Encouraging Good Processes Without the Need for Good Answers: Reinforcement Learning for LLM Agent Planning. *arXiv preprint arXiv:2508.19598* (2025).
 - [32] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.
 - [33] Hwijoon Lim, Juncheol Ye, Sangeetha Abdu Jyothi, and Dongsu Han. 2024. Accelerating model training in multi-cluster environments with consumer-grade gpus. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 707–720.
 - [34] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. 2025. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239* (2025).
 - [35] Han Lu, Zichen Liu, Shaopan Xiong, Yancheng He, Wei Gao, Yanan Wu, Weixun Wang, Jiashun Liu, Yang Li, Haizhou Zhao, Ju Huang, Siran Yang, Xiaoyang Li, Yijia Luo, Zihui Liu, Ling Pan, Junchi Yan, Wei Wang, Wenbo Su, Jiamang Wang, Lin Qu, and Bo Zheng. 2025. Part II: ROLL Flash – Accelerating RLVR and Agentic Training with Asynchrony. *arXiv preprint arXiv:2510.11345* (2025).
 - [36] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. 2025. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv preprint arXiv:2503.21620* (2025).
 - [37] Michael Luo, Sijun Tan, Roy Huang, Ameen Patel, Alpav Ariyak, Qingyang Wu, Xiaoxiang Shi, Rachel Xin, Colin Cai, Maurice Weber, Ce Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. 2025. DeepCoder: A Fully Open-Source 14B Coder at O3-mini Level. <https://>

- //pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51. (2025). Notion Blog.
- [38] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. 2025. GUI-R1 : A Generalist R1-Style Vision-Language Action Model For GUI Agents. *arXiv preprint arXiv:2504.10458* (2025).
- [39] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.
- [40] Aashiq Muhamed, Oscar Li, David Woodruff, Mona Diab, and Virginia Smith. 2024. Grass: Compute efficient low-memory llm training with structured sparse gradients. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 14978–15003.
- [41] NVIDIA Corporation. 2024. NVIDIA Multi-Process Service (MPS) Documentation. <https://docs.nvidia.com/deploy/mps/index.html>. (2024).
- [42] OpenPipe. 2025. Serverless RL. (2025). <https://openpipe.ai/blog/serverless-rl>
- [43] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2025. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE Press, 118–132. <https://doi.org/10.1109/ISCA59077.2024.00019>
- [44] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Shuo Yang, Yang Wang, Miryung Kim, Yongji Wu, Yang Zhou, Jiarong Xing, Joseph E. Gonzalez, Ion Stoica, and Harry Xu. 2025. ConServe: Fine-Grained GPU Harvesting for LLM Online and Offline Co-Serving. *arXiv preprint arXiv:2410.01228* (2025).
- [45] Ruoyu Qin, Weiran He, Weixiao Huang, Yangkun Zhang, Yikai Zhao, Bo Pang, Xinran Xu, Yingdi Shan, Yongwei Wu, and Mingxing Zhang. 2025. Seer: Online Context Learning for Fast Synchronous LLM Reinforcement Learning. *arXiv preprint arXiv:2511.14617* (2025).
- [46] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2024. Mooncake: A kv-cache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* (2024).
- [47] Haoran Qiu, Anish Biswas, Zihan Zhao, Jayashree Mohan, Alind Khare, Esha Choukse, Íñigo Goiri, Zeyu Zhang, Haiying Shen, Chetan Bansal, Ramachandran Ramjee, and Rodrigo Fonseca. 2025. ModServe: Modality- and Stage-Aware Resource Disaggregation for Scalable Multimodal Model Serving. In *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC 2025)*. Association for Computing Machinery, New York, NY, USA.
- [48] Mrinal Rawat, Ambuj Gupta, Rushil Goomer, Alessandro Di Bari, Neha Gupta, and Roberto Pieraccini. 2025. Pre-Act: Multi-Step Planning and Reasoning Improves Acting in LLM Agents. *arXiv preprint arXiv:2505.09970* (2025).
- [49] Amir Sarfi, Benjamin Thérien, Joel Lidin, and Eugene Belilovsky. 2025. Communication Efficient LLM Pre-training with SparseLoCo. (2025). arXiv:cs.LG/2508.15706 <https://arxiv.org/abs/2508.15706>
- [50] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).
- [51] SGLang Team. 2025. SGLang: Fast Serving Framework for Large Language Models. <https://github.com/sgl-project/sglang>. (2025). Version 0.4.
- [52] Zelei Shao, Vikranth Srivatsa, Sanjana Srivastava, Qingyang Wu, Alpay Ariyak, Xiaoxia Wu, Ameen Patel, Jue Wang, Percy Liang, Tri Dao, Ce Zhang, Yiyang Zhang, Ben Athiwaratkun, Chenfeng Xu, and Junxiong Wang. 2025. Beat the long tail: Distribution-Aware Speculative Decoding for RL Training. *arXiv preprint arXiv:2511.13841* (2025).
- [53] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [54] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. 2025. Laminar: A Scalable Asynchronous RL Post-Training Framework. *arXiv preprint arXiv:2510.12633* (2025).
- [55] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv:2409.19256* (2024).
- [56] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. verl: Volcano Engine Reinforcement Learning for LLM. <https://github.com/volcengine/verl>. (2024).
- [57] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [58] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. *arXiv preprint arXiv:2010.03768* (2021).
- [59] Joykirat Singh, Raghav Magazine, Yash Pandya, and Akshay Nambi. 2025. Agentic Reasoning and Tool Integration for LLMs via Reinforcement Learning. *arXiv preprint arXiv:2505.01441* (2025).
- [60] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
- [61] The Terminal-Bench Team. 2025. Terminal-Bench: A Benchmark for AI Agents in Terminal Environments. (2025). <https://github.com/laude-institute/terminal-bench>
- [62] Thinking Machines AI. 2025. Tinker. <https://thinkingmachines.ai/tinker/>. (2025). Accessed: 2026-02.
- [63] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache cache in the wild: characterizing and optimizing KVCache cache at a large cloud provider. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference (USENIX ATC '25)*. USENIX Association, USA, Article 28, 18 pages.
- [64] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Zihe Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. 2025. Reinforcement Learning Optimization for Large-Scale Learning: An Efficient and User-Friendly Scaling Library. *arXiv preprint arXiv:2506.06122* (2025).
- [65] Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, Yang Li, Zhongwen Li, Shirong Lin, Jiashun Liu, Zenan Liu, Tao Luo, Dilxat Muhtar, Yuanbin Qu, Jiaqiang Shi, Qinghui Sun, Yingshui Tan, Hao Tang, Runze Wang, Yi Wang, Zhaoguo Wang, Yanan Wu, Shaopan Xiong, Binchen Xu, Xander Xu, Yuchi Xu, Qipeng Zhang, Xixia Zhang, Haizhou Zhao, Jie Zhao, Shuaibing Zhao, Baihui Zheng, Jianhui Zheng, Suhang Zheng, Yanni Zhu, Mengze Cai, Kerui Cao, Xitong Chen, Yue Dai, Lifan Du, Tao Feng, Tao He, Jin Hu, Yijie Hu, Ziyu Jiang, Cheng Li, Xiang Li, Jing Liang, Xin Lin, Chonghuan Liu, ZhenDong Liu, Zhiqiang Lv, Haodong Mi, Yanhu Mo, Junjia Ni, Shixin Pei, Jingyu Shen, XiaoShuai Song, Cecilia Wang, Chaofan Wang, Kangyu Wang, Pei Wang,

- Tao Wang, Wei Wang, Ke Xiao, Mingyu Xu, Tiange Xu, Nan Ya, Siran Yang, Jianan Ye, Yaxing Zang, Duo Zhang, Junbo Zhang, Boren Zheng, Wanxi Deng, Ling Pan, Lin Qu, Wenbo Su, Jiamang Wang, Wei Wang, Hu Wei, Minggang Wu, Cheng Yu, Bing Zhao, Zhicheng Zheng, and Bo Zheng. 2026. Let It Flow: Agentic Crafting on Rock and Roll, Building the ROME Model within an Open Agentic Learning Ecosystem. *arXiv preprint arXiv:2512.24873* (2026).
- [66] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Yuchun Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems. *arXiv preprint arXiv:2401.17644* (2025).
- [67] Zhuang Wang, Zhaozhuo Xu, Jingyi Xi, Yuke Wang, Anshumali Shrivastava, and TS Eugene Ng. 2025. {ZEN}: Empowering Distributed Training with Sparsity-driven Data Synchronization. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 537–556.
- [68] Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. 2025. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 28489–28503.
- [69] Tianyuan Wu, Lunxi Cao, Yining Wei, Wei Gao, Yuheng Zhao, Dakai An, Shaopan Xiong, Zhiqiang Lv, Ju Huang, Siran Yang, Yinghao Yu, Jiamang Wang, Lin Qu, and Wei Wang. 2025. RollMux: Phase-Level Multiplexing for Disaggregated RL Post-Training. *arXiv preprint arXiv:2512.11306* (2025).
- [70] Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z. Morley Mao, Arvind Krishnamurthy, and Ion Stoica. 2025. RLBoost: Harvesting Preemptible Resources for Cost-Efficient Reinforcement Learning on LLMs. *arXiv preprint arXiv:2510.19225* (2025).
- [71] Bingquan Xia, Bowen Shen, Cici, Dawei Zhu, Di Zhang, Gang Wang, Hailin Zhang, Huaqiu Liu, Jiebao Xiao, Jinhao Dong, Liang Zhao, Peidiana Li, Peng Wang, Shihua Yu, Shimao Chen, Weikun Wang, Wenhan Ma, Xiangwei Deng, Yi Huang, Yifan Song, Zihan Jiang, Bowen Ye, Can Cai, Chenhong He, Dong Zhang, Duo Zhang, Guoan Wang, Hao Tian, Haochen Zhao, Heng Qu, Hongshen Xu, Jun Shi, Kainan Bao, Kai Fang, Kang Zhou, Kangyang Zhou, Lei Li, Menghang Zhu, Nuo Chen, Qiantong Wang, Shaohui Liu, Shicheng Li, Shuhao Gu, Shuhuai Ren, Shuo Liu, Sirui Deng, Weiwei Zhuang, Weiwei Lv, Wenyu Yang, Xin Zhang, Xing Yong, Xing Zhang, Xingchen Song, Xinze Xu, Xu Wang, Yihan Yan, Yu Tu, Yuanxuan Tian, Yudong Wang, Yue Yu, Zhenru Lin, Zhichao Song, and Zihao Yue. 2025. MiMo: Unlocking the Reasoning Potential of Language Model – From Pretraining to Posttraining. *arXiv preprint arXiv:2505.07608* (2025).
- [72] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. 2025. Aegaeon: Effective GPU pooling for concurrent LLM serving on the market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 1030–1045.
- [73] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. 2020. AntMan: Dynamic scaling on GPU clusters for deep learning. In *USENIX OSDI*.
- [74] Jiarong Xing, Yifan Qiao, Simon Mo, Xingqi Cui, Gur-Eyal Sela, Yang Zhou, Joseph Gonzalez, and Ion Stoica. 2025. Towards Efficient and Practical GPU Multitasking in the Era of LLM. *arXiv preprint arXiv:2508.08448* (2025).
- [75] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [76] David H. Yang, Mohammad Mohammadi Amiri, Tejaswini Pedapati, Subhjit Chaudhury, and Pin-Yu Chen. 2025. Sparse Gradient Compression for Fine-Tuning Large Language Models. (2025). *arXiv:cs.LG/2502.00311* <https://arxiv.org/abs/2502.00311>
- [77] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798* (2025).
- [78] Chenxuan Yao, Feifan Liu, Yuchong Hu, Zhengyu Liu, Xijue Zheng, and Wenxiang Zhou. 2025. LowDiff: Efficient Frequent Checkpointing via Low-Cost Differential for High-Performance Distributed Training Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, 1113–1126.
- [79] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. 2023. DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-like Models at All Scales. *arXiv preprint arXiv:2308.01320* (2023).
- [80] Minchen Yu, Rui Yang, Chaobo Jia, Zhaoyuan Su, Sheng Yao, Tingfeng Lan, Yuchen Yang, Yue Cheng, Wei Wang, Ao Wang, and Ruichuan Chen. 2025. lambdaScale: Enabling Fast Scaling for Serverless Large Language Model Inference. *arXiv preprint arXiv:2502.09922* (2025).
- [81] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476* (2025).
- [82] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiye Cao, Ke Bao, Ion Stoica, Harry Xu, and Ying Sheng. 2025. Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving. *arXiv preprint arXiv:2505.04021* (2025).
- [83] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2025. {BlitzScale}: Fast and Live Large Model Autoscaling with O(1) Host Caching. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 275–293.
- [84] Ruiqi Zhang, Daman Arora, Song Mei, and Andrea Zanette. 2025. SPEED-RL: Faster Training of Reasoning Models via Online Curriculum Learning. *arXiv preprint arXiv:2506.09016* (2025).
- [85] Yihao Zhao, Jiadun Chen, Peng Sun, Lei Li, Xuanzhe Liu, and Xin Jin. 2025. SeaLLM: Service-Aware and Latency-Optimized Resource Sharing for Large Language Model Inference. *arXiv preprint arXiv:2504.15720* (2025).
- [86] Yihao Zhao, Yuanqiang Liu, Yanghua Peng, Yibo Zhu, Xuanzhe Liu, and Xin Jin. 2022. Multi-resource interleaving for deep learning training. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 428–440.
- [87] Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. 2025. Act Only When It Pays: Efficient Reinforcement Learning for LLM Reasoning via Selective Rollouts. *arXiv preprint arXiv:2506.02177* (2025).
- [88] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating

- prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [89] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. 2025. StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation. *arXiv preprint arXiv:2504.15930* (2025).
- [90] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, et al. 2025. Optimizing {RLHF} training for large language models with stage fusion. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 489–503.

Appendix

A Rollout Concurrency Profiling

ROSE profiles and caps rollout concurrency on dedicated rollout GPUs to avoid excessive KV cache (KVC) memory pressure. Figure 14 shows the rollout throughput of Qwen3-8B with a 32K context length under different per-GPU batch sizes. Throughput increases with concurrency up to a batch size of 16, after which it saturates. Increasing concurrency beyond this point increases rollout latency due to memory contention and KVC fragmentation, which in turn degrades effective throughput. Therefore, unless otherwise specified, we cap the maximum number of concurrent rollout requests per dedicated rollout GPU at 16.

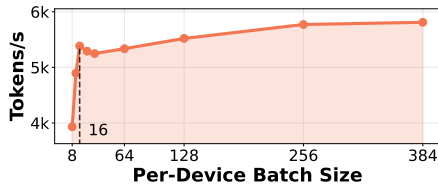


Figure 14. The system throughput with different per-device batch sizes. [Qwen3-8B/32K]

B Spot instance trace

We extract the spot-instance traces for the 8B model from Seg.B in Figure 8(a) and for the 32B model from Seg.B in Figure 9 of the RLBoost paper [70]. Figure 15 shows the variations in the number of preemptible and reserved GPUs over time. In the experiments, we used these traces to evaluate RLBoost’s performance.

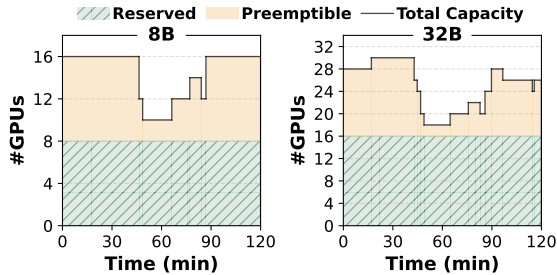


Figure 15. RLBoost trace used for 8B and 32B models.

C Sensitivity to caching lease

We set the lease time for prefix cache of rollout in the memory sharing policy. Table 5 evaluates how the prefix cache lease time affects rollout efficiency and serving SLOs for Qwen3-8B. The rollout time is largely insensitive to the lease time, suggesting that longer cache persistence provides limited additional benefit for rollouts in this setting. In contrast, lease time has a clear impact on serving tail latency: since

the Dual-SLO admission controller is designed to enforce SLO thresholds rather than explicitly minimize tail latency, a longer lease can keep more rollout KVC resident and reduce GPU memory headroom for bursty serving traffic, inflating P99 latency. Consequently, a moderate lease time offers the best trade-off, retaining most prefix cache of rollouts benefits while provide scheduling flexibility to prevent rollout prefix-cache residency from degrading serving SLOs.

Table 5. [Co-Serve Executor]. The impact of prefix-cache lease time on rollout and serving SLO.

Lease (s)	Avg Rollout Time (s)	TTFT P99 (ms)	TPOT P99 (ms)
10	496.3	338.11	136.13
20	497.1	334.71	135.28
50	492.2	371.10	140.70
100	502.1	491.90	148.10

D Sensitivity to Serving Traffic.

Serving GPUs can be repurposed for rollouts because serving demand fluctuates over time, leaving transient compute and memory headroom. To understand how ROSE behaves as this headroom shrinks, we scale the serving request arrival density and measure both rollout efficiency (average rollout time) and serving QoS (TTFT/TPOT P99). Table 6 shows that higher serving density increases resource contention, which in turn prolongs rollouts and degrades serving tail latency.

Two additional observations stand out. First, TTFT is more sensitive to increasing load than TPOT, suggesting that contention primarily hurts the prefill, while per-token generation is comparatively less affected. Second, the larger model exhibits more stable rollout time as serving density increases, but its serving tail latencies still rise with load, indicating that compute/memory interference persists and must be managed by the co-serving executor. Overall, the results confirm the expected trade-off under cooperative elasticity: as serving load grows, ROSE gradually reduces effective rollout capacity while keeping serving performance degradation bounded rather than causing sharp SLO violations.

Table 6. [Co-Serve Executor]. The impact of serving traffic density on rollout and serving SLO.

Model	Density	Avg Rollout Time (s)	TTFT P99 (ms)	TPOT P99 (ms)
Qwen3-8B	1	496.3	338.1	136.1
	1.5	511.7	380.2	149.0
	2	569.9	459.1	150.1
Qwen3-32B	1	960.1	837.5	398.1
	1.5	977.7	870.2	421.3
	2	989.9	899.1	441.2